SIMD Advantage Profiling

Andrew Zheng | Advanced Computer Systems | Project 1 | October 17, 2026

Purpose and principal findings

This project measures when compiler-generated SIMD improves single-thread numeric kernels and when wider vectors offer little benefit. Elementwise multiplication, AXPY and dot product were tested with float32 and float64 across cache-sized and larger-than-cache arrays. Scalar, GCC-default auto-vectorized and CPU-targeted native builds use the same kernel source.

For N=1,024, native SIMD gives 6.00x speedup for float multiply and 8.69x for float AXPY. At N=33,554,432, those speedups shrink to 1.23x and 1.59x. Double multiply and AXPY show approximately 1x speedup at the largest size. Dot product remains close to scalar because its observed vector multiplication feeds a serial accumulation chain. Thus, a vectorization report alone does not establish a useful speedup.

Measured platform and recorded environment

Item	Recorded value
Processor	AMD Ryzen 7 9850X3D; 8 cores, 16 logical CPUs
Caches	L1D: 48 KiB/core; L2: 1 MiB/core; L3: 96 MiB shared
ISA exposed to guest	x86-64, SSE, AVX2, AVX-512 and FMA
OS / virtualization	Ubuntu 24.04.5 LTS; Microsoft WSL2
Kernel	6.18.33.2-microsoft-standard-WSL2
Compiler	GCC 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1)
Execution policy	One benchmark thread; final ISA sweep pinned to logical CPU 0
SMT / power snapshot	SMT active; Windows Balanced power scheme
Frequency policy	Linux cpufreq policy unavailable; operating frequency not logged

The environment snapshot was collected after the experiments and is not proof that every earlier run used identical settings. The virtual CPU affinity does not reserve a physical core, disable its SMT sibling, or control Windows scheduling. BIOS and power-policy continuity between separate datasets was not independently verified.

Dataset scope

The analysis uses 2,016 final ISA-sweep trials, 1,008 alignment/tail trials, 896 stride trials, 84 bandwidth-calibration trials and 504 counter trials: 4,508 recorded batch measurements. The earlier two-build sweep is superseded by the final three-build sweep for baseline comparisons. Raw data and processed summaries accompany this report.

Methodology and vectorization evidence

Kernels are compiled separately from the timing harness, without link-time optimization. All kernels use -O3 and -ffp-contract=off. The scalar build adds -march=native -fno-tree-loop-vectorize -fno-tree-slp-vectorize; the default build has no -march/-mtune override; the native-auto build adds -march=native. The common harness uses -O2 and the correctness checker uses -O0.

Timing uses std::chrono::steady_clock. Repetition counts double until a batch reaches approximately 30 ms (or the repetition cap); seven batch-average trials are retained. Two warmup calls precede each trial. Allocation, initialization, validation and CSV output are excluded from the timed batch. The ISA sweep rotates kernel/build ordering and covers powers of two from 2^10 through 2^25. Alignment and stride runners vary configuration order and pin to an allowed logical CPU.

Kernel	Operation	FLOP/element	Useful bytes/element
Multiply	z[i] = x[i] * y[i]	1	3T
AXPY	y[i] += a * x[i]	2	3T
Dot	sum += x[i] * y[i]	2 (convention)	2T

T is the element size in bytes. Footprint counts unique arrays: 3NT for multiply and 2NT for AXPY/dot. Useful bandwidth counts algorithmic reads/writes, not physical DRAM traffic. GFLOP/s = N x FLOP/element / seconds / 10^9; GiB/s = useful bytes / seconds / 2^30. Speedup is the ratio of scalar and vector-build median times at matching kernel, type and N.

Multiply uses small positive integer-valued inputs. AXPY alternates a=+0.5 and -0.5 to restore y without timed reset copies; its intermediate result is checked too. Dot uses x[i]=(i mod 7)-3 and y[i]=1+(i mod 5), whose product sum over 35 elements is zero. The bounded exact reference avoids float accumulation overflow/precision drift in this test. Correctness checks cover N=0,1,5,8,16,17,27,32,33,1000 for both types.

Evidence from compiler reports and inspected assembly

The default compiler report identifies 16-byte vectorization; native multiply/AXPY assembly shows 64-byte ZMM operations. The scalar designation means scalar arithmetic, even when instructions use XMM registers. No FMA is present in the inspected AXPY loop, consistent with disabled contraction. Selected native float instructions are:

```
multiply: vmovups zmm1, [x]    vmulps zmm0, zmm1, [y]
          vmovups [z], zmm0
dot:      vmulps zmm1, zmm5, [y]
          vaddss xmm0, xmm0, xmm1  ... repeated into xmm0
```

The dot loop vectorizes products but extracts lanes into dependent scalar additions. Its limited speedup therefore does not contradict the compiler report. The stride kernel also checks stride==1 before its SIMD path; non-unit strides branch to vmulss/vmulsd scalar loops. Full dumps are retained as evidence; only explanatory excerpts are shown here.

Throughput across working-set sizes

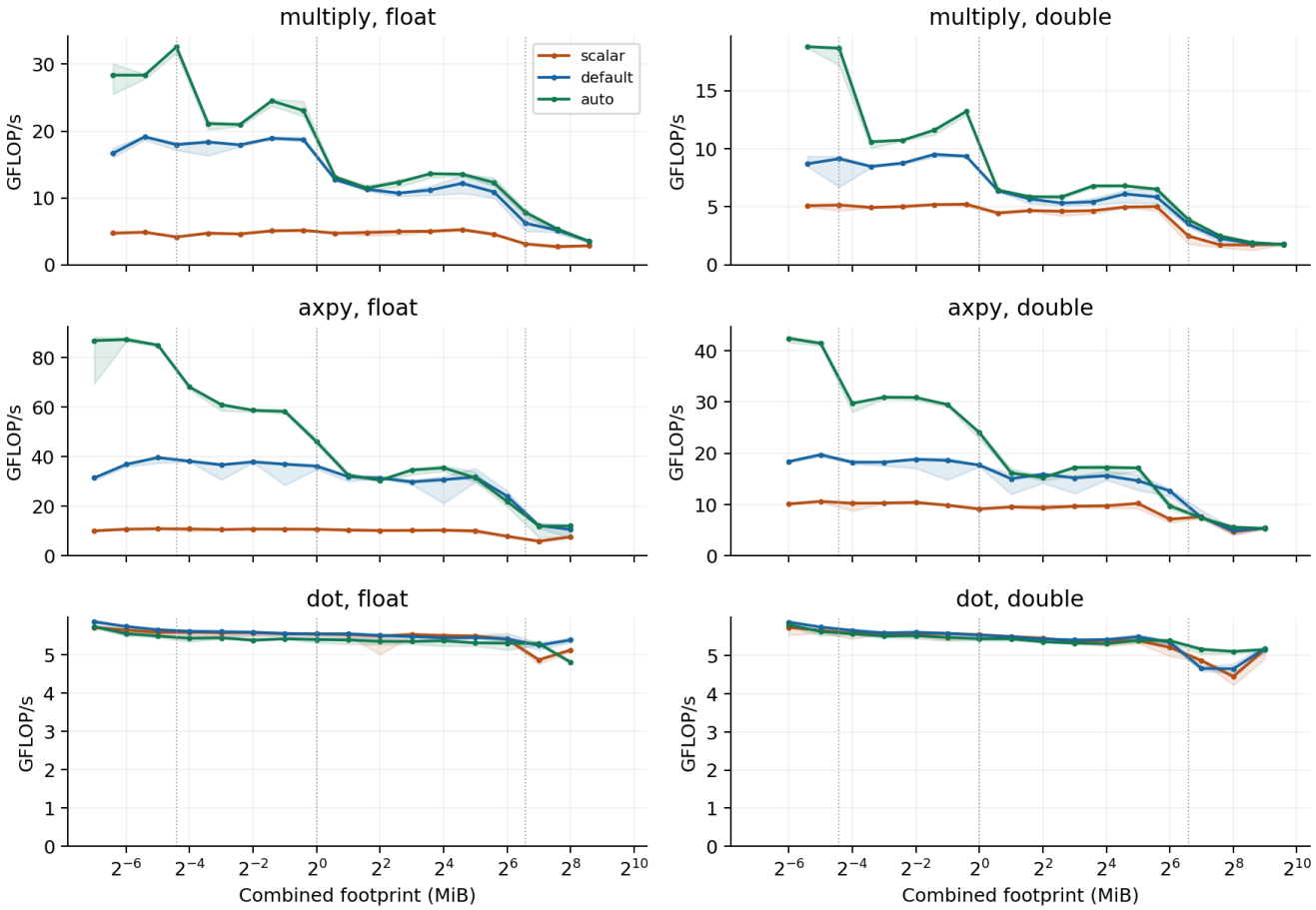


Figure 1. Median throughput for all three kernels and builds. Shading shows each configuration's observed min-max range. Vertical dotted lines mark nominal L1D, L2 and L3 capacities (48 KiB, 1 MiB, 96 MiB).

At small footprints, multiply and AXPY benefit from processing many independent elements per vector instruction. Float also offers twice as many lanes as double at the same vector width. As footprints grow, data movement becomes increasingly important and the throughput curves fall or converge. Cache boundaries are approximate: associativity, shared capacity, prefetching and other activity can shift transitions.

At the largest N, native float multiply reaches 3.479 GFLOP/s and double reaches 1.743 GFLOP/s. AXPY reaches 11.967 and 5.309 GFLOP/s, respectively. These kernels have different arithmetic intensities and write behavior, so their GFLOP/s values should not be interpreted as equal memory traffic or equal fractions of compute peak.

The dot kernel stays near its scalar baseline. Serial reduction dependency and floating-point evaluation order limit the benefit of packed multiplication. The experiment deliberately does not enable fast-math reassociation to transform the reduction into a different numerical algorithm.

Speedup, locality and data type

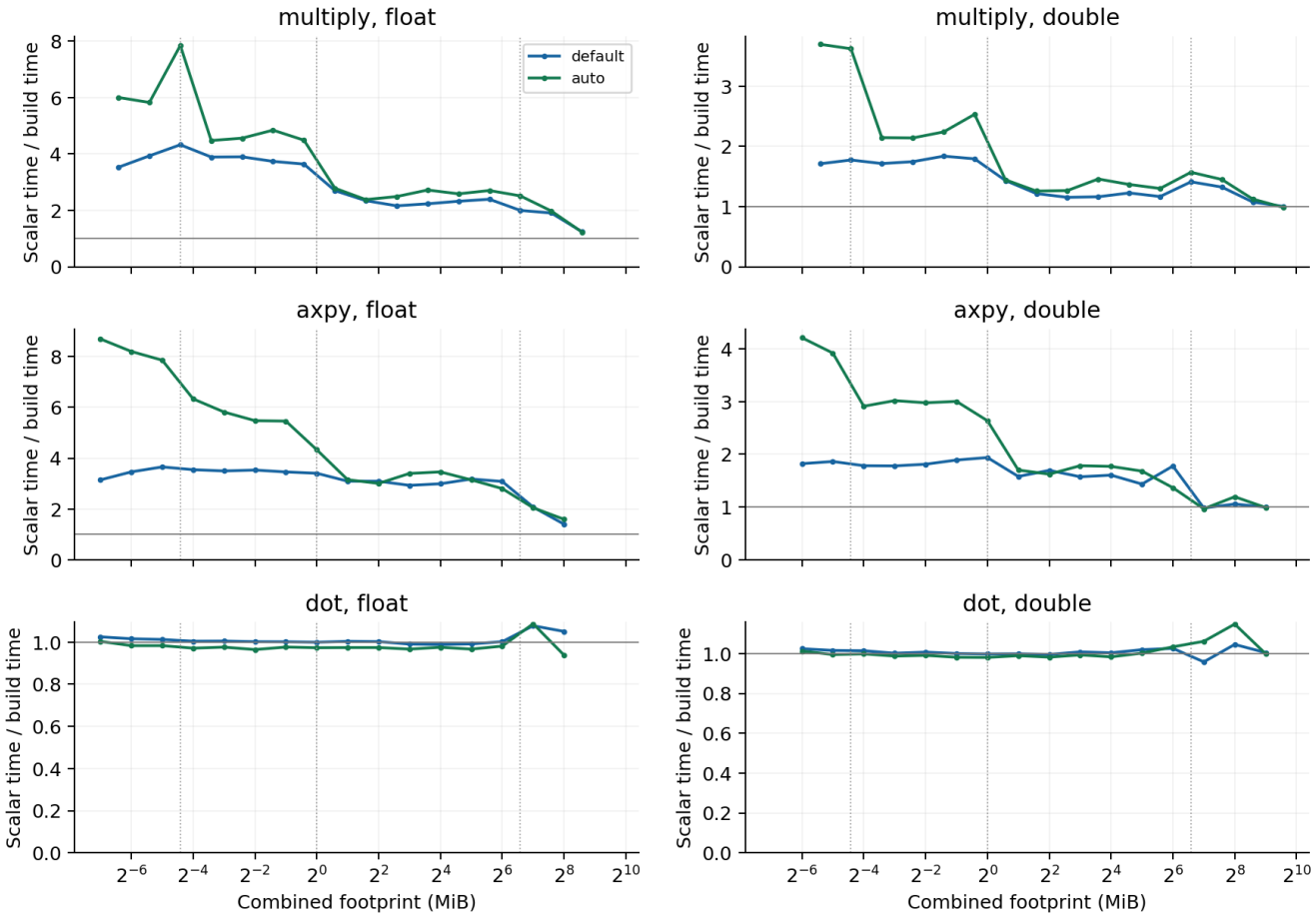


Figure 2. Ratios of median scalar time to median default/native time. The horizontal line is 1x. Cache markers use the combined footprint. Ratios are point estimates; timing variability is retained in Figure 1 and the raw trials.

Native float multiply speedup decreases from 6.00x at N=1,024 to 1.23x at N=33,554,432; native float AXPY decreases from 8.69x to 1.59x. Double multiply falls from 3.70x to 0.98x and double AXPY from 4.21x to 0.99x. Increasing vector width therefore gives its clearest benefit where arithmetic and instruction overhead matter, not uniformly across memory regimes.

At equal N, double requires twice the storage and useful bytes, while a vector holds half as many elements. Both effects contribute to data-type differences. Comparisons plotted by footprint help locate cache transitions, but equal-footprint cases have different N and are not identical instruction-count workloads.

The 128-bit default build can match or slightly exceed the native build on large arrays. At the largest size its multiply throughput is 3.510 GFLOP/s float and 1.773 GFLOP/s double. These modest differences are insufficient to claim a general advantage for narrower vectors; bandwidth pressure and run-to-run variation can outweigh vector-width benefits.

Alignment and tail handling

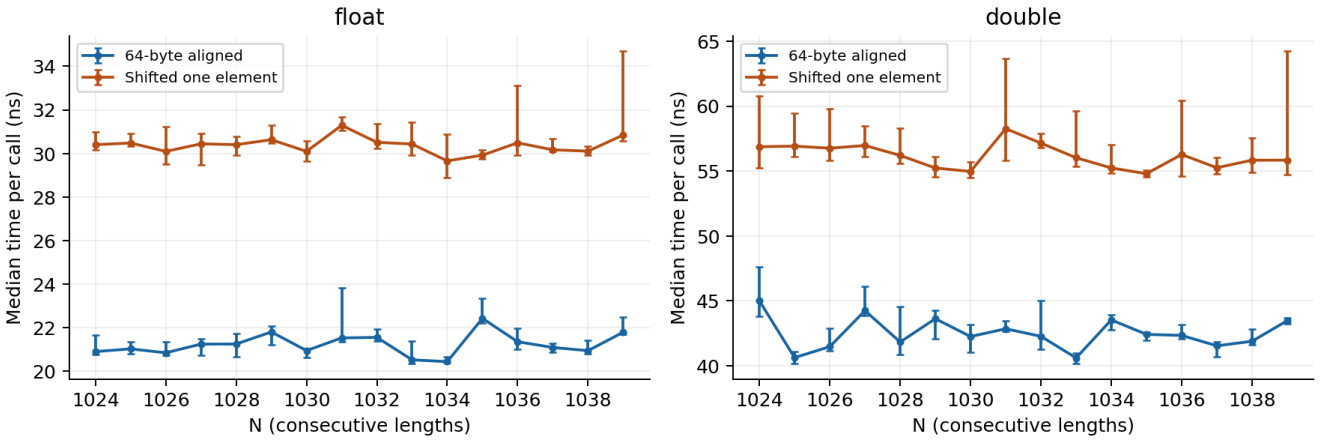


Figure 3. Native multiply at consecutive lengths 1,024-1,039. Points are seven-trial medians; bars show min-max. Arrays start on a 64-byte boundary or one naturally aligned element beyond it.

All three arrays receive the same offset: 4 bytes for float or 8 bytes for double. This preserves natural type alignment. Consecutive lengths cover every remainder for 16-float vectors and twice every remainder for 8-double vectors. Larger cases N=65,536 and 65,537 test the same address offsets with a different footprint.

Native multiply	Aligned time	Shifted time	Shifted / aligned
float, N=1,024	20.89 ns	30.40 ns	1.455
double, N=1,024	44.98 ns	56.87 ns	1.264
float, N=65,536	3.155 us	3.147 us	0.997
double, N=65,536	10.842 us	11.409 us	1.052

The inspected code uses unaligned-capable vmovups/vmovupd in both address cases. It has no scalar alignment peel or masked tail in these multiply loops. The main loop processes 64-byte vectors, then an optional 32-byte block, followed by up to seven scalar floats or three scalar doubles. Runtime alias checks can also select a scalar fallback.

At small N, shifted addresses are slower in these measurements; cache-line-crossing vector accesses are a plausible contributor, not an isolated proven cause. Tail cost is not monotonic: aligned float N=65,537 was faster than 65,536 (2.562 versus 3.155 us). This anomaly cautions against assigning every adjacent-size difference to remainder overhead. Guard checks detect out-of-range writes and untouched-output corruption but do not prove absence of out-of-range reads.

Stride and useful bandwidth

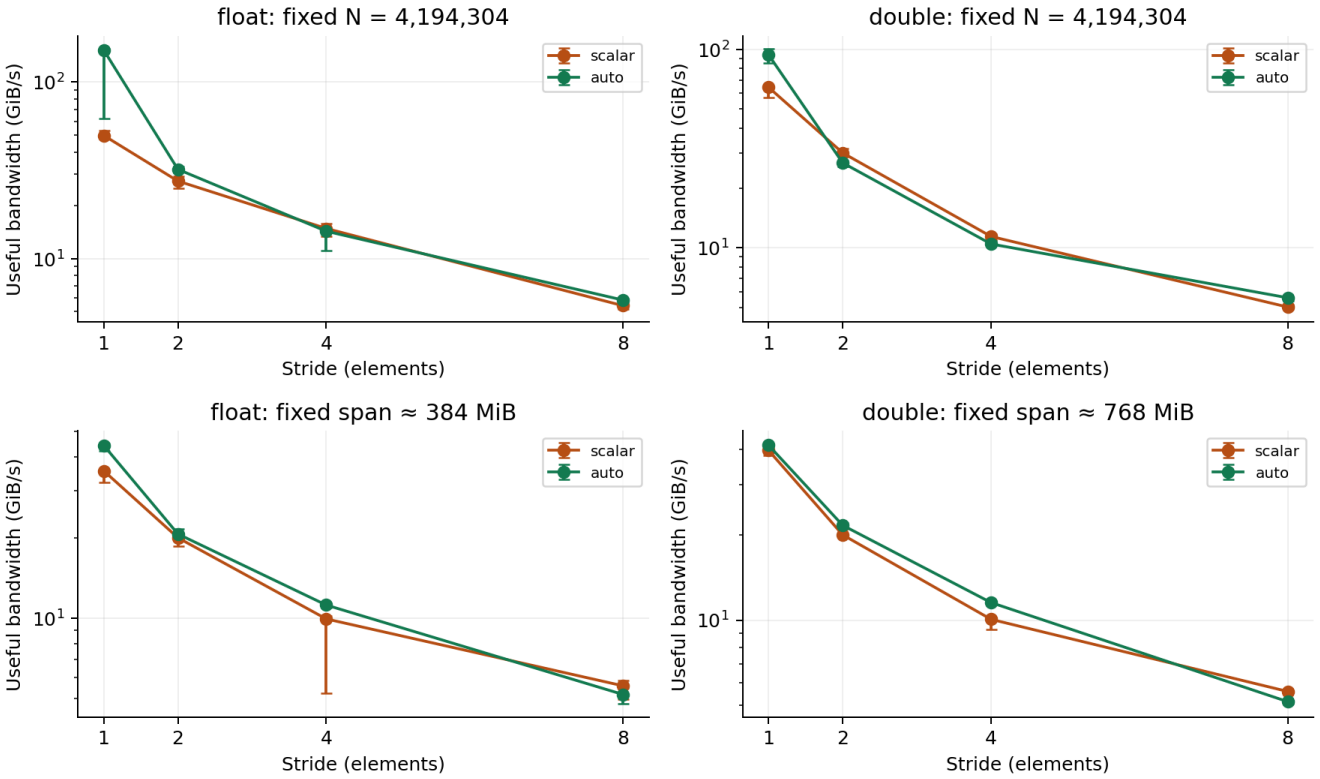


Figure 4. Largest stride-control groups, with medians and min-max bars. Top: fixed operation count while address span grows. Bottom: approximately fixed combined address span while the number of operations decreases.

Strides 1,2,4,8 access element i*stride. For fixed N, the useful work stays constant while the span becomes 3*((N-1)*stride+1)*T. The fixed-span control holds N*stride constant; actual spans differ by at most seven elements per array. Useful bandwidth remains 3NT/time, rather than counting skipped elements as useful work.

In the largest fixed-span native cases, useful bandwidth falls from 44.28 to 5.18 GiB/s for float and from 41.46 to 5.15 GiB/s for double between strides 1 and 8. The decline is consistent with using fewer elements from each fetched cache line. It is not a direct measurement of physical DRAM traffic or proof of a specific prefetch failure.

Assembly establishes a second effect: only unit stride reaches the vector loop; other tested strides use scalar multiplication. Thus the stride-1 to stride-2 change combines vectorization loss with a changed access pattern. No gather/scatter implementation is used. The scalar baseline and fixed-span control help interpret the loss, but do not fully isolate every memory-system mechanism.

Bandwidth calibration and roofline model

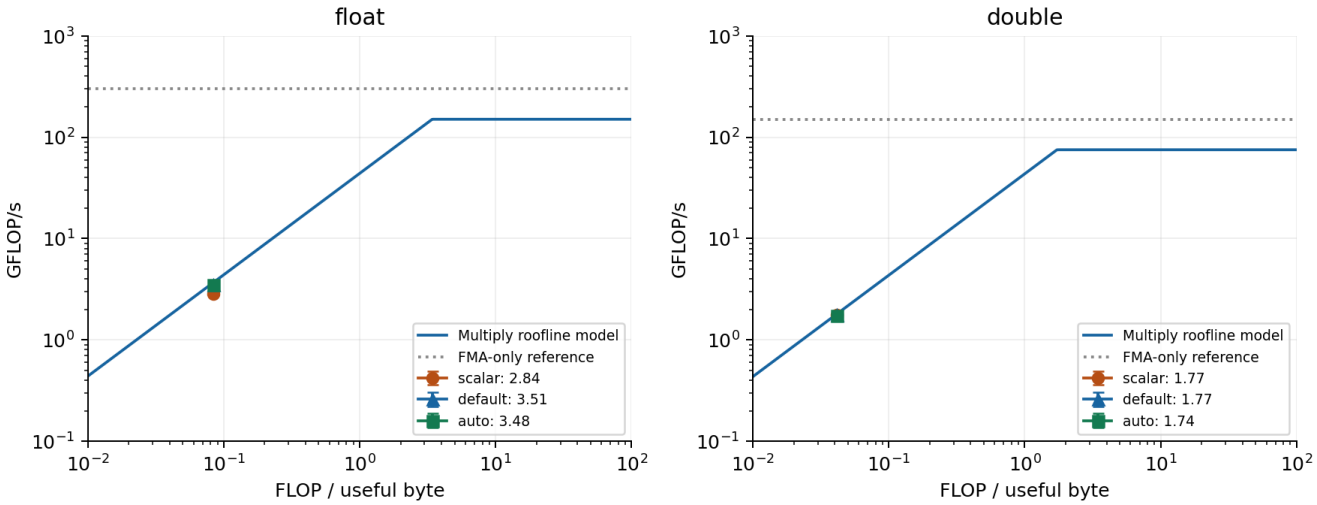


Figure 5. Largest-size multiply measurements from the final ISA sweep (384 MiB float, 768 MiB double). Error bars show min-max. The solid model combines measured useful bandwidth with a multiply-only compute estimate at an assumed 4.7 GHz.

Quantity	float	double
Useful bandwidth reference	40.93 GiB/s	40.56 GiB/s
Multiply arithmetic intensity	1/12 FLOP/byte	1/24 FLOP/byte
Memory performance reference	3.662 GFLOP/s	1.815 GFLOP/s
Measured native multiply	3.479 GFLOP/s	1.743 GFLOP/s
Measured / memory reference	95.0%	96.1%

The calibration uses an independent streaming-add loop with two reads and one ordinary cached write. Its reference is the median of two run medians at N=67,108,864, with seven trials per run. Largest-size totals are 768 MiB float and 1,536 MiB double. Inspected assembly confirms ZMM vaddps/vaddpd and cached stores. The reference uses the large-size plateau rather than the highest smaller-footprint rate.

P = min(P_compute, B_useful * I_useful). GiB/s is converted to decimal GB/s before multiplication. At the modeled 4.7 GHz, two 512-bit multiplies/cycle give 150.4 GFLOP/s float or 75.2 double. A conventional FMA-only reference doubles those numbers to 300.8/150.4; multiply itself performs one FLOP per lane. This architectural estimate uses the listed base clock [1] and desktop Zen 5 throughput research [2], not measured operating frequency.

The measured multiply points are near the empirical memory reference and far below the arithmetic ceiling, supporting a memory-bound interpretation. Useful bandwidth absorbs cache/write-allocation effects; it is not a physical-byte counter result or an absolute hardware bound. Cache-resident points are excluded from this streaming roofline because they can exceed its memory slope.

Hardware cycles per element

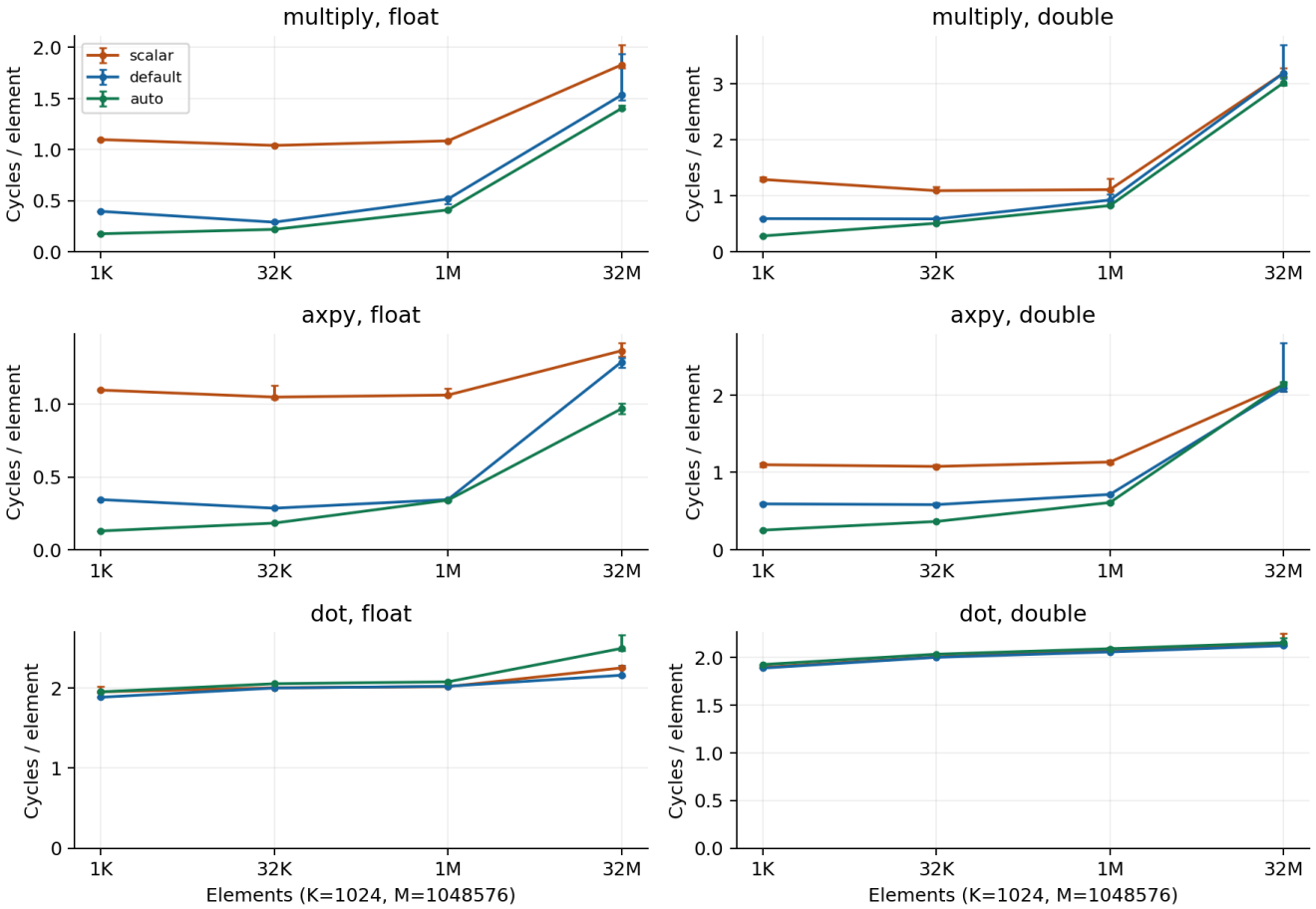


Figure 6. Hardware user-mode cycles per element from a separate counter run. Points are medians of seven trials; bars are min-max. K=1,024 and M=1,048,576 elements.

A successful perf availability check was followed by a dedicated perf_event_open harness [3]. Each trial creates a fresh disabled, pinned CPU_CYCLES event for the calling thread, excludes kernel/hypervisor execution, enables it around the kernel batch and disables it before validation. CPE = raw counted cycles / (calls*N). No assumed clock is used to calculate CPE.

All 504 accepted measurements report running_time/enabled_time=1.0, so no multiplex scaling was applied. Counts include batch-loop, function-call and timer overhead; initialization, calibration, warmups, correctness checks and output are excluded. These are guest-exposed hardware-event measurements under WSL, not bare-metal isolation.

For N=1,024, float multiply uses 1.097 cycles/element scalar versus 0.175 native; double multiply uses 1.290 versus 0.281. At the largest size, double AXPY is nearly unchanged at 2.126 scalar and 2.139 native. Counter and timing sweeps are separate datasets, so small cross-run differences should not be interpreted as contradictions or converted into precise effective clock rates.

Conclusions, limitations and reproducibility

What the experiments establish

SIMD is most beneficial for independent, contiguous work that can exploit cache locality. Wider vectors do not guarantee proportional speedup. Large-array double multiply provides the required negative example: inspected code contains SIMD, yet the final timing sweep gives 0.98x native/scalar speedup. Its throughput is close to the independently calibrated memory reference. Dot product is a second caution: packed multiplication coexists with a serial reduction. Non-unit runtime stride is a third: the vectorized code path is bypassed entirely.

Limitations and honest interpretation

Early informal observations included other background application load; they are not treated as a controlled CPU-stress study or used as the final baseline. Background load was not continuously logged. The current snapshot shows Balanced power and SMT enabled, but historical BIOS/power continuity is unverified. Linux frequency controls are unavailable in this WSL guest. Core affinity reduces migration but does not isolate host scheduling or sibling activity.

Several configurations have large timing ranges. All observations are retained; medians and min-max ranges are used without claiming confidence intervals. Alignment, stride and calibration datasets were collected separately. Cache-residency boundaries are nominal. No cache-miss or physical-memory-traffic counters were collected, so explanations involving those mechanisms are reasoned interpretations, not direct counter measurements.

Reproducing and auditing the work

The repository should retain build/run scripts, original run directories, compiler reports, assembly evidence, raw measurements, processed tables and plotting scripts. This report package includes the selected raw data and reproducible figure/PDF generators. The final step is to commit the completed deliverables and verify that the repository access meets the instructor's submission requirements.

References

[1] AMD, Ryzen 7 9850X3D specifications: https://shop-us-en.amd.com/amd-ryzen-7-9850x3d-processor/
[2] Alexander Yee, Zen5's AVX512 Teardown, original instruction-throughput experiments: https://www.numberworld.org/blogs/2024_8_7_zen5_avx512_teardown/
[3] Linux man-pages, perf_event_open(2): https://man7.org/linux/man-pages/man2/perf_event_open.2.html